# HTML5, CSS3 & JavaScript

## The Three Core Skills of Front-End Web Development

A beginner-friendly guide: what each one is, what it does, and how they work together

**Structure** **Style** **Behavior**

OVERVIEW

# How the Web Is Built

Every website you visit is made from three technologies working together. Think of building a house: **HTML5** is the structure (walls, rooms, doors), **CSS3** is the design (paint, furniture, layout), and **JavaScript** is the electricity (lights, switches, and everything that reacts when you interact).

| Skill | Role | Analogy | File type |
|---|---|---|---|
| **HTML5** | Structure and content | The skeleton of the house | .html |
| **CSS3** | Presentation and layout | Paint, decor, and floor plan | .css |
| **JavaScript** | Behavior and logic | Electricity and automation | .js |

## What each one answers

- **HTML5** answers: *What is on the page?* (headings, text, images, forms, videos)
- **CSS3** answers: *How does it look?* (colors, fonts, spacing, responsive layouts)
- **JavaScript** answers: *What does it do?* (clicks, validation, animations, data loading)

> **Key idea:** Browsers read your HTML to build the page, apply your CSS to style it, and run your JavaScript to make it interactive. Learn them in this order: HTML, then CSS, then JavaScript.

## A tiny example of all three together

```
<button id="btn">Click me</button>      <!-- HTML: structure -->

button { background: #1572B6;           /* CSS: style */
         color: white; padding: 10px; }

btn.onclick = () => alert("Hello!");    // JS: behavior
```

# 1 HTML5

HyperText Markup Language, version 5: the structure of every web page

## What is it?

HTML is a **markup language**, not a programming language. It uses **tags** to label content so the browser knows what each piece is: a heading, a paragraph, a link, an image. HTML5 is the modern version and added meaningful layout tags, built-in audio and video, better forms, and powerful browser APIs.

## Basic page skeleton

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>My First Page</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>Hello, World!</h1>
  <p>This is my first web page.</p>
  <script src="app.js"></script>
</body>
</html>
```

## Essential tags

| Tag | Purpose | Example |
|---|---|---|
| **<h1> to <h6>** | Headings, from largest to smallest | <h1>Title</h1> |
| ** ** | Paragraph of text | Hello |
| **<a>** | Link to another page | <a href="/about">About</a> |
| **<img>** | Image (always add alt text) | <img src="cat.jpg" alt="A cat"> |
| **<ul> / <ol> / <li>** | Bulleted, numbered lists | <ul><li>Item</li></ul> |
| ** / ** | Generic block and inline containers | ... |
| **<form> / <input>** | Collect user input | <input type="email"> |
| **<table>** | Tabular data | <tr><td>Cell</td></tr> |

## What is new in HTML5

**1. Semantic elements.** Tags that describe their meaning, which helps accessibility and SEO:

```
<header>  Site logo and navigation   </header>
<nav>     Menu links                  </nav>
<main>
  <article> A blog post </article>
  <aside>   Sidebar     </aside>
</main>
<footer>  Copyright info              </footer>
```

**2. Native audio and video** with no plugins like Flash:

```
<video src="movie.mp4" controls width="400"></video>
<audio src="song.mp3" controls></audio>
```

**3. Smarter form inputs** with built-in validation:

```
<input type="email" required>
<input type="date">
<input type="number" min="1" max="10">
<input type="range"> <input type="color">
```

**4. More features:**

- **<canvas>** for drawing graphics and simple games
- **Web Storage** (localStorage, sessionStorage) to save data in the browser
- **Geolocation API** to find a user's location (with permission)
- **SVG** support for sharp, scalable graphics

> **Best practices:** Use semantic tags, always write alt text on images, use only one <h1> per page, and keep your HTML clean and properly indented.

# 2 CSS3

Cascading Style Sheets, version 3: the design and layout of the page

## What is it?

CSS tells the browser **how HTML elements should look**. It controls colors, fonts, spacing, sizes, positions, animations, and how the page adapts to phones, tablets, and desktops. CSS3 is the modern version, split into modules, and it brought rounded corners, shadows, gradients, transitions, Flexbox, and Grid.

## How a CSS rule works

```
selector {
  property: value;
}

h1 {
  color: #1572B6;
  font-size: 32px;
  text-align: center;
}
```

## Three ways to add CSS

- **External** (best): <link rel="stylesheet" href="style.css">
- **Internal:** inside a <style> tag in the page head
- **Inline:** style="color:red" directly on an element (use sparingly)

## Common selectors

| Selector | Targets | Example |
|---|---|---|
| **Element** | All tags of that type | p { ... } |
| **Class (.)** | Elements with that class | .card { ... } |
| **ID (#)** | One unique element | #header { ... } |
| **Descendant** | Elements inside others | nav a { ... } |
| **Pseudo-class** | A state of an element | a:hover { ... } |

## The Box Model

Every element is a box made of four layers, from inside out: **content**, **padding** (space inside the border), **border**, and **margin** (space outside). Adding `box-sizing: border-box;` makes sizing much easier to predict.

## Powerful CSS3 features

**Flexbox**: lay items out in a row or column and align them easily.

```
.menu {
  display: flex;
  justify-content: space-between;   /* spread items out */
  align-items: center;              /* center vertically */
  gap: 16px;
}
```

**Grid**: build two-dimensional layouts with rows and columns.

```
.gallery {
  display: grid;
  grid-template-columns: repeat(3, 1fr);   /* 3 equal columns */
  gap: 20px;
}
```

**Responsive design** with media queries, so your site fits every screen:

```
@media (max-width: 600px) {
  .gallery { grid-template-columns: 1fr; }   /* 1 column on phones */
}
```

**Transitions, animations, and visual effects:**

```
.btn {
  background: linear-gradient(135deg, #1572B6, #38BDF8);
  border-radius: 8px;
  box-shadow: 0 4px 12px rgba(0,0,0,.2);
  transition: transform .3s;
}
.btn:hover { transform: scale(1.05); }

@keyframes fade { from {opacity:0} to {opacity:1} }
```

> **Best practices:** Use external stylesheets, name classes clearly, design mobile-first, use CSS variables (`--main-color`) for reusable colors, and avoid overusing !important.

# 3 JavaScript

The programming language of the web: the behavior and logic of the page

## What is it?

JavaScript is a real **programming language** that runs in the browser. It lets pages respond to clicks, validate forms, update content without reloading, fetch data from servers, and create animations and games. With Node.js it also runs on servers, so one language can power both the front end and back end.

## Core building blocks

```
// Variables
let name = "Sasank";       // can change
const year = 2026;         // cannot be reassigned

// Functions
function greet(person) {
  return "Hello, " + person + "!";
}
const add = (a, b) => a + b;     // arrow function

// Conditions and loops
if (year > 2025) { console.log("Modern era"); }
for (let i = 1; i <= 3; i++) { console.log(i); }

// Arrays and objects
const skills = ["HTML", "CSS", "JS"];
const user = { name: "Sasank", age: 22 };
skills.map(s => s.toUpperCase());
```

## Data types

String, Number, Boolean, Null, Undefined, Object (including Arrays and Functions), and Symbol/BigInt for special cases.

## Making pages interactive: the DOM

The **DOM** (Document Object Model) is the browser's live tree of your HTML. JavaScript can read and change it:

```
<h1 id="title">Hello</h1>
<button id="btn">Change</button>

<script>
  const title = document.getElementById("title");
  const btn   = document.querySelector("#btn");

  btn.addEventListener("click", () => {
    title.textContent = "You clicked the button!";
    title.style.color = "tomato";
    title.classList.add("highlight");
  });
</script>
```

## Modern JavaScript (ES6+)

- **let / const** instead of var for safer variables
- **Arrow functions** and **template literals:** `` `Hi ${name}` ``
- **Destructuring** and the **spread operator** (...) for easy data handling
- **Modules** (import / export) to organize code into files
- **Promises and async/await** for asynchronous tasks

## Fetching data from an API

```
async function loadUsers() {
  try {
    const res  = await fetch("https://api.example.com/users");
    const data = await res.json();
    console.log(data);
  } catch (err) {
    console.error("Something went wrong:", err);
  }
}
loadUsers();
```

> **Best practices:** Prefer const, use clear names, handle errors with try/catch, keep functions small, and test in the browser console (press F12).

PUTTING IT ALL TOGETHER

# A Complete Mini Project: Theme Toggle

This small example uses all three skills: **HTML** builds the page, **CSS** styles it, and **JavaScript** adds a dark mode button.

## index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Theme Toggle</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <main class="card">
    <h1>Hello, Developer!</h1>
    <p>Click the button to switch themes.</p>
    <button id="toggle">Toggle Dark Mode</button>
  </main>
  <script src="app.js"></script>
</body>
</html>
```

## style.css

```
body { font-family: sans-serif; background: #f1f5f9; color: #0f172a;
       display: grid; place-items: center; min-height: 100vh; margin: 0;
       transition: background .3s, color .3s; }
body.dark { background: #0f172a; color: #f1f5f9; }
.card { text-align: center; padding: 40px; border-radius: 12px;
        box-shadow: 0 6px 20px rgba(0,0,0,.15); }
button { padding: 10px 20px; border: 0; border-radius: 8px;
         background: #1572B6; color: #fff; cursor: pointer; }
```

## app.js

```
document.getElementById("toggle").addEventListener("click", () => {
  document.body.classList.toggle("dark");
});
```

NEXT STEPS

# Learning Roadmap & Quick Reference

| Stage | Focus | Practice project |
|---|---|---|
| **1. HTML5** | Tags, semantic layout, forms, media | Personal profile page |
| **2. CSS3** | Box model, Flexbox, Grid, responsive design | Responsive landing page |
| **3. JavaScript basics** | Variables, functions, arrays, loops | Calculator, to-do list |
| **4. DOM and events** | Selecting and changing elements | Quiz app, image slider |
| **5. Modern JS** | ES6+, fetch, async/await, modules | Weather app using an API |
| **6. Next level** | Git, Tailwind CSS, React, Node.js | Full portfolio website |

## Quick comparison

| | HTML5 | CSS3 | JavaScript |
|---|---|---|---|
| **Type** | Markup language | Style language | Programming language |
| **Purpose** | Structure content | Design and layout | Interactivity and logic |
| **Syntax** | <tag>content</tag> | selector { property: value; } | let x = 5; function(){} |
| **Can it make decisions?** | No | Limited (media queries) | Yes (if, loops, functions) |
| **Learn it** | First | Second | Third |

## Free resources to keep learning

- **MDN Web Docs** (developer.mozilla.org): the best free reference for all three
- **web.dev** and **freeCodeCamp**: free guided courses and practice
- **Browser DevTools** (press F12): inspect any site and experiment live

> **Remember:** Do not just read, build. Making small projects every week is the fastest way to master HTML5, CSS3, and JavaScript.